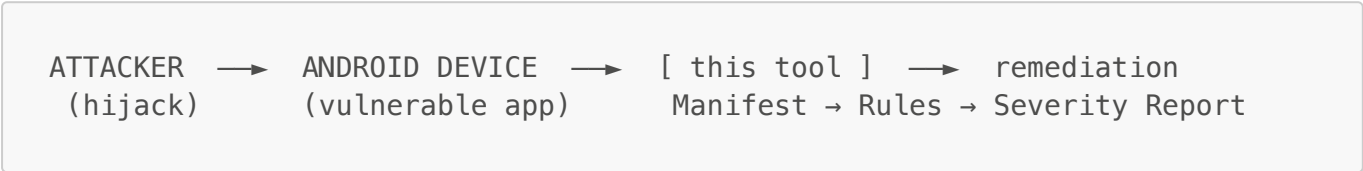


SMD Manifest Auditor

Defensive static auditor for the **QR Code & Deep Link Hijacking in Android** project (Security of Mobile Devices, Assignment 3 — Phase 3).

It is the defensive half of the project: where the offensive PoC *exploits* a poorly configured deep link, this tool *detects* the insecure configuration before it ships. It obtains an app's `AndroidManifest.xml`, applies a small set of rules, and prints a **severity report** (human-readable and JSON).



What it checks

The rules map directly onto the four families in the project plan, plus a few bonus application-level hardening checks.

ID	Family	Severity	Detects
EX001	Exported components	HIGH (CRITICAL for provider)	An exported component with no android:permission that isn't a launcher or a verified App Link — any app can invoke it (intent injection).
EX002	Exported components	MEDIUM	A component with an <code>intent-filter</code> but no explicit android:exported (implicitly exported / ambiguous on Android 12+).
DL001	Deep links	HIGH	A custom-scheme deep link (e.g. <code>smdpoc://</code>) — custom schemes have no origin verification, so any app can register the same scheme (scheme collision).
DL002	Deep links	MEDIUM	An <code>http/https</code> deep link missing android:autoVerify — an unverified App Link other apps can intercept.
BR001	Sensitive deep links	CRITICAL	A sensitive component (oauth / callback / login / pay / admin / ...) reachable via an unverified BROWSABLE deep link — a direct hijack / account-takeover path.
APP001	App hardening (bonus)	HIGH	<code>android:debuggable="true"</code> .
APP002	App hardening (bonus)	LOW	<code>android:allowBackup</code> enabled (or defaulted).

ID	Family	Severity	Detects
APP003	App hardening (bonus)	MEDIUM	<code>android:usesCleartextTraffic="true"</code> .

A correctly built **verified App Link** (`https + autoVerify="true"`) is the recommended secure pattern, so it is treated as safe and is **not** flagged — this keeps the false-positive rate low. The bonus app-level checks can be turned off with `--no-extras`.

Requirements

- **Python 3.8+** — the auditor itself uses only the standard library.
- For the **device modes** only, the Android SDK tools must be on your **PATH**:
 - `adb` (platform-tools)
 - `apkanalyzer` (cmdline-tools/latest/bin)
- For running the tests: `pip install -r requirements.txt` (installs `pytest`).

No installation step is needed for the auditor — just run `auditor.py`.

Usage

The tool takes exactly one input source.

1. Local manifest file (no device needed)

Use this for the demo and for grading. Point it at a decoded `AndroidManifest.xml`:

```
python3 auditor.py --manifest samples/vulnerable_AndroidManifest.xml
python3 auditor.py --manifest samples/hardened_AndroidManifest.xml
```

2. One installed app on a connected device / emulator

```
python3 auditor.py --package com.smd.poc
```

This runs `adb shell pm path`, pulls the base APK, and decodes its manifest with `apkanalyzer manifest print`.

3. Every third-party app on the device

```
python3 auditor.py --all # third-party packages
python3 auditor.py --all --system # include system packages too
```

Common options

Option	Meaning
<code>--json</code> <code>report.json</code>	Also write the full report as JSON.
<code>--device SERIAL</code>	Target a specific adb device (when several are connected).
<code>--no-extras</code>	Disable the bonus app-level hardening checks (APP001–003).
<code>--no-color</code>	Plain text output (colour is auto-disabled when piped).
<code>--fail-on LEVEL</code>	Exit code 1 if a finding at this severity or higher exists. Default HIGH . Use NEVER to always exit 0.

Use in CI

The exit code makes it usable as a build gate (Phase 0 CI):

```
# fails the pipeline if any HIGH or CRITICAL issue is present
python3 auditor.py --manifest app/src/main/AndroidManifest.xml --fail-on
HIGH
```

The severity report

Every run ends with a severity report:

SEVERITY REPORT

```
CRITICAL  2
HIGH      6
MEDIUM   3
LOW       1
INFO      0
```

```
Risk score : 100 / 100  -> CRITICAL RISK
```

Severity levels (highest impact first):

- **CRITICAL** — directly exploitable for hijack / account takeover (e.g. a sensitive component on an unverified deep link, or an exported provider).
- **HIGH** — a strong exposure that meaningfully enables the attack (custom scheme, exported component without a guard, debuggable build).
- **MEDIUM** — a weakness that helps an attacker or is ambiguous (unverified App Link, implicit export, cleartext traffic).
- **LOW** — hardening gap with limited direct impact (backup enabled).

- **INFO** — informational only.

Risk score is the sum of per-finding weights (CRITICAL = 40, HIGH = 20, MEDIUM = 8, LOW = 3), capped at 100. The score maps to an **overall rating**:

Score	Rating
0	CLEAN
1–19	LOW
20–49	MEDIUM
50–79	HIGH
80–100	CRITICAL

JSON output (`--json`) has the same information, structured for tooling:

```
{
  "tool": "smd-manifest-auditor",
  "version": "1.0",
  "generated": "2026-05-23T16:57:00+00:00",
  "targets": [
    {
      "package": "com.smd.poc",
      "source": "manifest:samples/vulnerable_AndroidManifest.xml",
      "summary": {
        "CRITICAL": 2, "HIGH": 6, "MEDIUM": 3, "LOW": 1, "INFO": 0,
        "total": 12, "risk_score": 100, "risk_rating": "CRITICAL"
      },
      "findings": [
        {
          "rule_id": "BR001",
          "family": "Sensitive deep links",
          "severity": "CRITICAL",
          "title": "Sensitive component is reachable via an unverified deep link",
          "component": "activity .0AuthCallbackActivity",
          "evidence": "name matches sensitive pattern, schemes=['smdpoc']",
          "description": "...",
          "recommendation": "..."
        }
      ]
    }
  ]
}
```

Demo (Phase 4 validation)

The `samples/` folder contains the two fixtures used to validate the tool:

```
# 1) the deliberately vulnerable app -> every planted issue is flagged
python3 auditor.py --manifest samples/vulnerable_AndroidManifest.xml
#    => CRITICAL RISK, 12 findings (exit code 1)

# 2) the same app after remediation -> clean
python3 auditor.py --manifest samples/hardened_AndroidManifest.xml
#    => CLEAN, 0 findings (exit code 0)
```

This is the "findings appear, then clear after the fix" loop from the architecture diagram ([remediation guidance](#)).

Tests

```
pip install -r requirements.txt
pytest -q
```

The suite asserts that each rule fires on the vulnerable fixture, that the hardened fixture is free of HIGH/CRITICAL findings, and that the launcher activity is never flagged as a false positive.

Project layout

```
auditor/
├── auditor.py           # the tool (stdlib only)
├── README.md           # this file
├── requirements.txt     # pytest (for the tests)
├── .gitignore
├── samples/
│   ├── vulnerable_AndroidManifest.xml # all planted issues
│   └── hardened_AndroidManifest.xml  # mitigations applied
└── tests/
    └── test_auditor.py      # pytest suite
```

Ethics

This is a **defensive** analysis tool. The accompanying offensive PoC and this auditor are for use only on the team's own deliberately vulnerable app, inside an isolated lab environment, for the coursework. Do not run the attack against apps you do not own or are not explicitly authorised to test.